Solution Architecture

Semantic Governance
Solution Architecture (MVP1 Prep)

Solution Architecture: Federated RAG 2.0 with Metaphactory & Gemini

This architecture creates an intelligent, self-optimizing system for managing enterprise knowledge. It establishes a dynamic link between abstract business concepts and concrete digital assets, powered by a component-based design that promotes scale and domain-specific learning.

Top-Down Knowledge Layer (Core & Domain Ontologies)

This is the foundation of the architecture. It's the central source of business truth, defining what the company is, how it operates, and what it knows.

- **Core Ontologies:** A foundational set of ontologies (Process, Service, Asset, Product, Party, Location, Time, Event, Issue) that provides a consistent, high-level vocabulary for the entire enterprise. All domain-specific ontologies will extend and reuse these core concepts.
- **Domain-Specific Ontologies:** For each business domain (e.g., Sales, Marketing, HR), an ontology is built on top of the core ontologies. This layer enriches the core concepts with domain-specific terms, relationships, and business rules, representing the organization's unique knowledge in a machine-readable format.
- **Metaphactory's Semantic Modeling Assistant:** This agent is used to author and manage the core and domain-specific ontologies. It provides a visual, user-friendly interface for knowledge engineers and subject matter experts to create and extend these top-down structures.

Bottom-Up Knowledge Layer (Digital Assets)

This layer represents the raw data and metadata from across the enterprise. It's the physical reality that the top-down layer seeks to model.

- **Schemas and Metadata:** This includes database tables, columns, business intelligence metrics, hierarchies, and other metadata from all digital assets (e.g., Salesforce, ERP, data lakes).
- **Digital Asset Inventory:** The schemas and metadata are automatically or semi-automatically ingested into a central repository (e.g., Data Plex in GCP), creating a comprehensive inventory of all digital assets.

Mapping Layer & Federated Agents

This is the most critical and intelligent part of the architecture, where the connection between the top-down and bottom-up layers is established and continuously improved.

- **Mapping Ontologies:** Each bottom-up schema (e.g., `users_table`, `sales_transactions`) will have its own dedicated mapping ontology. This ontology formally defines the relationships (e.g., `isRepresentedBy`, `hasAttribute`) between specific top-down ontological classes and the bottom-up schema elements. This granular approach supports polymorphic meaning.
- **Metaphactory's Search & Discover Agent:** This agent acts as a federated interface. It uses the top-down and mapping ontologies to enable users to search for data using business terms. It translates a business query like "Find customer emails for the Q3 sales campaign" into the technical query required to retrieve `users_table.email_address` from the data source.
- **Domain-Specific AI Agents:** An individual AI agent is assigned to each business domain (Sales, Marketing, HR, etc.). These agents are responsible for two core functions:
    1. **Mapping at Scale:** They continuously learn and propose new mappings between top-down concepts and bottom-up schemas, accelerating the manual process. They identify patterns and suggest relationships that human experts might miss.
    2. **Extending Knowledge:** They analyze unstructured data and user queries within their domain to propose extensions to the domain-specific ontologies, vocabularies, and taxonomies, ensuring the knowledge layer remains comprehensive and up-to-date.

Federated RAG 2.0 with Virtual SLMs

This layer explains how the system interacts with a single large language model (Gemini) to provide grounded, context-aware responses without the need for multiple, costly specialized language models (SLMs).

- **Vector Databases per Domain:** A separate vector database is created for each business domain. These are not "virtual SLMs" in the traditional sense, but they serve as a virtual, domain-specific knowledge cache. They store vectorized representations of:
    1. The top-down and mapping ontologies for that domain.
    2. Conversational queries and their successful mappings.
    3. Positive and negative feedback from users.
- **The Single LLM (Gemini):** A single, powerful LLM acts as the core reasoning engine. It doesn't need to be specialized for each domain because it has the knowledge layer to rely on.
- **Federated RAG 2.0 Workflow:**
    1. **Query from User:** A user submits a query to the conversational AI interface (e.g., "What are our most profitable products?").
    2. **Query Decomposition & Domain Routing:** The conversational AI and a separate routing agent (a simple rule-based agent or a specialized LLM call) analyze the query and identify the most relevant business domains (e.g., Sales, Finance).
    3. **Federated Retrieval:** The query is sent to the vector databases of the

relevant domains. The system retrieves the most relevant semantic mappings, scenarios, and terms. This is the **Retrieval** step of RAG.

4. **Enriched Context for LLM:** The retrieved information (the mappings, relevant ontology classes, and past successful queries) is combined with the user's original question to create a highly specific and grounded prompt.
5. **Generation & Response:** Gemini generates a response based on this enriched, grounded prompt. The response can then be reverse-engineered to verify the mappings and update the vector database with positive feedback.

Recursive Learning & Governance

The system is designed for continuous improvement and strong governance.

- **Recursive Learning:** Every successful query and mapping is stored as positive feedback in the domain-specific vector database. When a new query is submitted, the system first consults its learned mappings, continuously improving its accuracy and speed. If a query fails, the negative feedback is captured, allowing the AI agents to propose corrections or extensions to the ontologies or mappings.
- **Component Design:** The architecture is modular. Each domain is a self-contained component with its own ontology, vector DB, and AI agents. This allows for scalability and independent development without impacting other domains.
- **Central Governance:** The core ontologies and the mapping process are governed by a central team. This ensures consistency and prevents data fragmentation. A governance dashboard tracks the health of each domain's knowledge layer, monitors mapping confidence scores, and flags areas for human intervention.
- **LookML Integration:** The metadata for the schemas is automatically generated from the LookML models in Looker, ensuring that the bottom-up layer is always in sync with the live analytics layer. This data is then consumed by the Metaphactory agents and vector databases.

Industry Alignment

Semantic Governance
(Alignment with Industry Best Practice)

This document outlines how our approach to building domain-specific knowledge—leveraging top-down ontologies mapped to bottom-up metadata via a process-centric model—aligns with the concepts and methodologies presented in the following two authoritative resources:

- Pliks et al., 2010, "Knowledge management and decision support in adaptive case management platforms" (pulled from IEEE - link below https://ieeexplore.ieee.org/abstract/document/7321628?signout=success)
- (try the link below you can't get thru): https://www.academia.edu/72947572/Knowledge_Management_and_Decision_Support_in_Adaptive_Case_Management_Platforms?email_work_card=view-paper

- Search-O1 Project, Enterprise Semantic Search Framework (https://search-o1.github.io/) (recommended by Rajender R Nednur)

Overview of Our Approach

Specifically, our approach:

- Combines top-down ontology engineering with bottom-up metadata extraction, reflecting the dual-layer knowledge modeling advocated by Pliks et al. (see Sections 2 and 5);
- Leverages a process-centric semantic model aligned with Pliks' emphasis on business process impact on ontology structure (Section 4);
- Incorporates iterative validation and refinement mechanisms consistent with Pliks et al.'s recommendations (Section 6);
- Implements semantic vector embeddings and unified ontology-schema mappings that enable semantic and conversational search, which aligns strongly with Search-O1's architecture and goals (as detailed in their Concepts and Architecture pages);
- Enforces modular subdomain decomposition and context filtering, vital in Search-O1 and supported by Pliks et al. for scalable, domain-aware knowledge management;
- Provides a knowledge fabric foundation that enables intelligent agents to operate under a consistent top-down knowledge topology at enterprise scale.

This combined framework delivers a resilient, explainable, and scalable semantic search and conversational agent environment, practical for complex domains such as sales operations.

Semantic Governance
(Alignment with Industry Best Practice)

Integration of Top-Down and Bottom-Up Ontologies

- Pliks et al. (Section 2, p. 3-5) highlight the critical value of integrating *expert-curated top-down ontologies* with *bottom-up extracted knowledge* to comprehensively capture enterprise semantics.
- Our approach mirrors this by utilizing a domain-specific sales operations ontology (top-down) mapped explicitly to metadata from digital asset schemas (bottom-up). This hybrid data model maintains semantic rigor while grounding ontology classes in practical, data-driven instances.

Process-Centric Semantic Modeling

- In Section 4 (p. 9-11), Pliks et al. emphasize modeling business processes and workflows to appropriately structure ontologies, improving alignment with operational semantics and data source contexts.
- Our architecture organizes sales operations into subdomains (lead management, pipeline, order management), using a process-centric model to clarify concept relationships and improve mapping accuracy—a direct application of these principles.

Iterative Mapping Validation and Refinement

- Section 6 (p. 14-16) describes the importance of continuous validation of ontology-to-data mappings supported by evidence and confidence metrics, suggesting iterative refinement to maintain semantic integrity.
- Our database captures mapping confidence scores, validation statuses, evidence texts, and training labels (positive/negative) to enable this ongoing refinement, ensuring conceptual accuracy and robustness.

Ontology-Driven Semantic Search

- Pliks et al. discuss how ontology-driven integration supports semantic querying and retrieval (Abstract, Section 7, p. 17-18). Semantic layers allow for *context-aware*, *concept-based search* transcending simple keyword matching.
- Our use of dense vector embeddings aligned with ontology vocabulary and mapping tables operationalizes semantic search, empowering intelligent agents to understand queries at a conceptual level rooted in the knowledge fabric.

Modularity and Scalability

- The paper advocates modular ontology design to accommodate the complexity of enterprise domains and simplify maintenance (Section 5, p. 12-13).
- Our design incorporates subdomain indexing and modular service areas to manage complexity modularly, in line with these recommendations.

Semantic Embeddings and Vector Databases

- Search-O1’s core architecture rests on transforming ontological and schema

knowledge into vector spaces to support semantic similarity search (Concepts page).
- Our use of vector embeddings for ontology entities and schema metadata directly implements this principle, enabling robust semantic retrieval crucial for enterprise-scale knowledge access.

Unified Ontology-Schema Mapping

- Search-O1 stresses mapping domain ontologies to heterogeneous metadata schemas for uniform enterprise insight and searchability (Architecture page).
- The mapping tables in our model reflecting entity correspondences, types, confidence, and evidence match this, providing organized, machine-readable links to unify disparate data assets.

Contextual Subdomain Filtering

- The project promotes sophisticated knowledge layer modularization where subdomains or domains can be scoped and filtered during query or agent interactions.
- Our subdomain index and process-centric model embody this concept, enabling agents to operate with semantic precision by focusing on the most relevant domain slices.

Intelligent Agents and Conversational Search

- Search-O1 envisions agents leveraging the knowledge fabric to power natural language search and complex workflows (Agent and Conversational Search demos).
- Our curated knowledge fabric, enriched with validated mappings, enables agents to perform contextually informed conversational search, consistent with their vision for scoping, reasoning, and retrieval.

Semantic Governance
(Alignment with Industry Best Practice)

Capability Map & Alignment with Best Practices

Capability	Supported By Our Approach	Corresponding Article Evidence
Comprehensive, accurate domain representation	Top-down ontology + bottom-up metadata mapping	Pliks et al. Sections 2, 5, 6
Context-aware, process-centric semantic relations	Process-centric domain modeling	Pliks et al. Section 4; Search-O1 modular domain design
Semantic vector search and retrieval	Entity embeddings with vector indexes	Search-O1 vector space architectures; Pliks et al. semantic querying
Iterative validation and refinement	Mapping confidence, labeling, and evidence tracking	Pliks et al. Section 6
Unified enterprise knowledge fabric	Schema-ontology mappings for digital asset consistency	Search-O1 unified ontology-schema architecture
Intelligent agent & conversational search support	Semantic layers plus mapping driven agent guidance	Search-O1 agent-based conversational frameworks

Primary Process Flows

Semantic Governance
(Primary Process Flows)

Ingestion & Initial Mapping

This phase describes how a new or updated data source is introduced into the system and how the initial mapping process is triggered.

1. **Source Schema Ingestion**: The process begins when a new data source schema is ingested or an existing one is updated. This event triggers the start of the lifecycle.
2. **Process Path Selection**: The system evaluates whether the ingested schema is completely new or an update to an existing one. This decision point, or exclusive gateway, directs the flow down one of two paths:
    - **Manual Mapping**: If it's a new schema, a **Knowledge Engineer** or Data Steward must manually initiate the mapping process. This approach ensures human oversight for complex, new data structures.
    - **Automated Mapping**: If it's an update, the process automatically begins an automated mapping routine. This path is used for efficiency when changes are minor and can be handled by AI agents.

Domain-Specific Mapping & Guidance

Regardless of whether the process is manual or automated, the core mapping work is assisted by specialized agents to ensure accuracy and context.

3. **Mapping Assistance**: Both the manual and automated paths lead to a sub-process where the **Metaphactory Platform** and its agents work to map the schema. This step is guided by a **Verizon Domain Agent**, a specialized AI agent trained on a specific domain. The domain agent provides critical context and guidance to ensure the mapping aligns with internal business knowledge.
4. **Mapping Execution**: The Metaphactory system, with the help of the domain agent, performs the actual mapping. It links the new or updated schema to existing ontologies and identifies new relations. The output of this stage is a mapped schema ready for further processing.

Parallel Post-Mapping Tasks

Once the core mapping is complete, the process branches into several parallel tasks to propagate the new knowledge throughout the system.

5. **Concurrent Updates**: A parallel gateway activates four concurrent service tasks to update and extend various components of the semantic ecosystem:
    - **Vector Database Update**: The new semantic embeddings and mappings are immediately used to update the **Vector Database**, ensuring that the knowledge graph is current for retrieval-augmented generation (RAG).
    - **Semantic Model Extension**: The core semantic models and ontologies are

extended with the new relationships, concepts, and mappings. This maintains a centralized and authoritative source of knowledge.

- **AI Model Training**: The new data is fed to the **AI Training System** to further refine the models, improving their ability to understand, classify, and generate insights based on the updated knowledge.
- **Monitoring Metrics Capture**: Key metrics related to the mapping process—such as success rates, time taken, and quality scores—are captured and fed to the **Monitoring & Dashboards** system for analysis.

Quality Assurance & Iteration

After all parallel tasks are complete, the system performs a critical quality check to ensure the integrity of the knowledge.

6. **Quality Assurance**: The concurrent tasks join at another gateway, and the process moves to a quality check. The system evaluates whether the mapping quality is satisfactory.
7. **Iterative Feedback Loop**: This is a key decision point in the process:
   - If the mapping is **not satisfactory**, the process is routed back to the **Manual Mapping** step. A Knowledge Engineer must then review and correct the issues, re-initiating the cycle.
   - If the mapping **is satisfactory**, the process proceeds to the final publishing stage.

Publishing & Release

This final phase ensures that the approved, high-quality knowledge is formally published and made available to the broader organization.

8. **Scheduled Publishing**: The process waits for a **weekly timer event**. This ensures that approved mappings are released in a controlled, predictable manner rather than immediately.
9. **Ontology Publishing**: When the timer triggers, the **Publishing & Release Management** system publishes the new and updated ontologies. This makes the approved knowledge available for consumption by data products, applications, and end-users.
10. **Completion**: The process concludes once the publishing is successfully completed.

Feature Breakdown

Semantic Governance
(Feature Breakdown)

Functional Features

Functional features describe what the system does to support each step of the process.

- **Schema & Metadata Ingestion**: The ability to import data schemas and associated metadata from various sources (e.g., APIs, databases, files) into the platform.
- **Automated Mapping Engine**: An AI-powered service that automatically identifies, evaluates, and proposes mappings between ingested data schemas and existing ontologies.
- **Manual Mapping Interface**: A user interface that allows Knowledge Engineers to manually review, edit, and create mappings.
- **AI Agent Guidance**: A system for specialized AI agents (like the Verizon Domain Agent) to provide context and suggest mappings for a given domain.
- **Ontology Extension Service**: A service that validates and applies new classes, properties, and relationships to the core semantic models and ontologies.
- **Vector Embedding Generation**: The capability to convert structured data and relationships into high-dimensional vectors for use in a vector database for Retrieval Augmented Generation (RAG).
- **AI Model Fine-Tuning**: A process that updates and trains large language models (LLMs) with new data and mappings to improve their performance.
- **Dashboard & Metrics**: A user interface that visualizes key performance indicators (KPIs) related to data quality, mapping progress, and system health.
- **Publishing & Release Workflow**: An automated or semi-automated system that publishes approved ontologies and mappings to a production environment on a set schedule.

Semantic Governance
(Feature Breakdown)

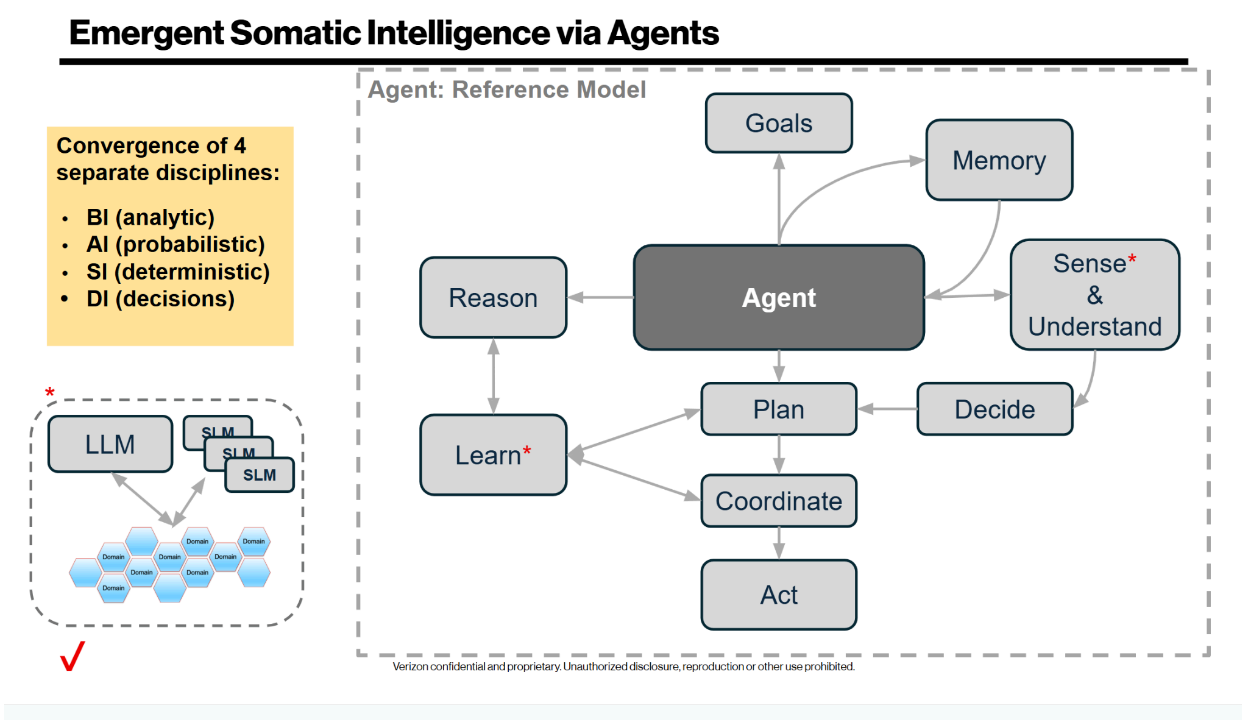
Non-Functional Features

Non-functional features describe how the system operates, focusing on its performance and reliability.

- **Scalability**: The system's ability to handle an increasing volume of schemas and mapping operations without a degradation in performance.
- **Performance**: The speed and responsiveness of the system, particularly for automated mapping, AI training, and data retrieval tasks.
- **Reliability**: The system's ability to operate continuously with minimal downtime, including robust error handling and recovery mechanisms.
- **Security & Access Control**: Features that ensure data integrity, confidentiality, and proper authorization, such as role-based access to the mapping interface and data encryption.
- **Usability**: The ease of use of the platform's interfaces, especially for Knowledge Engineers and Data Stewards responsible for mapping and quality assurance.
- **Interoperability**: The capability of the system to integrate seamlessly with various external data sources and downstream applications.

Building Knowledge

Semantic Governance
(Somatic Model for Building Knowledge)



Functional Breakdown

Top-Down Knowledge Knowledge Assembly

- Core Ontologies
- DS Ontologies
- DS Search Agent
- DS Mapping Agent
- DS Vector DB
- Orchestration Agent

SG Operational Dashboard

- Demand Management (Intake)
- Lifecycle Management (all 5 phases)
- Service Level Management
  - 🌕 Inbound
  - 🌕 Processing
  - 🌕 Outbound
- Issue Management

Semantic Governance
(Somatic Model for Building Knowledge)

- Capacity Management
- Business Continuity Management

AI Monitoring & Training Dashboard

- **For EACH Agent…**
- Monitoring
- Measurement
- Training (Learn = Pos + Neg)
- Transparency (Decide)
- Recommendations (Act)

SG Scheduler

- Ingestion (initial, ongoing)
- Mapping at Scale (Bottom-Up)
- Extending Knowledge at Scale
- Batch Configuration
- Orchestration & Threading

Prompt Engineering

- Sense
  - 🌕 Intake
  - 🌕 Reverse Engineering
- Understand
  - 🌕 Relevancy (Domains)
  - 🌕 Relevancy (Persona)
- Act
  - 🌕 Orchestration
  - 🌕 Search & Discover
  - 🌕 Generate Results
    - Mapping Recommendations (Digital Schemas & Filtering)
    - Respond (tailoring and refining the results)

Vector DB Design

Semantic Governance
(Vector DB Design Pattern)

The following is a design for building vector databases to capture domain-specific context for guiding a sales operations agent—one constrained by a domain-specific ontology and tasked with mapping ontology classes to schema ontologies for digital assets—requires an architecture that integrates several nuanced components and processes.

Core Architecture Overview

The database should be a hybrid system, combining:

- Vector DB (Domain) A vector store for semantic retrieval
- Mapping Ontology (Schema) - Relational/graph storage for explicit mappings and relations
- Metadata and embedding support for context-rich queries

Key Components and Data Flows

- **Ontology Entities Table**: Stores classes and properties from both the sales operations domain ontology and schema ontologies for digital assets. Includes fields for entity type, metadata, embedding (vector representation), and links to sub-domains.
- **Embeddings Index:** Each ontology class/property (e.g., “Lead,” “Account,” “Opportunity,” “Contract” in sales ops) is embedded into a vector space, using semantic embedding models suitable for domain-specific alignment. **This supports the approximate nearest neighbor (ANN) search for mapping discovery.**
- **Mapping Table:** Explicit links between domain ontology classes and schema ontology classes representing digital assets, with fields for:
    - Source class
    - Target class
    - Mapping confidence (numeric or categorical)
    - Mapping type (equivalent, broader, narrower) (relation from Upper Ontology)
    - Validation status and history.
- **Sub-domain Context Index**: Sales operations consists of sub-domains (e.g., lead management, quoting, order fulfillment, pricing, sales analytics). Each sub-domain has distinct classes that may overlap or intersect. The index tracks associations, context vectors, and provides scope filtering for querying and agent guidance.

Semantic Governance
(Vector DB Design Pattern)

Flow of Activities

1. **Entity Extraction:** Upon agent request, sales ops ontology classes are extracted and embedded. Metadata (name, description, relationships, sub-domain context) is stored.
2. **Semantic Mapping:** Matches between domain ontology classes and digital asset schema classes are proposed using vector similarity (semantic matching), then refined/ranked with syntactic, lexical, and structural features.
3. **Mapping Validation:** High-confidence mappings are validated and persisted with evidence; ambiguous mappings are flagged for review. Mappings are linked with sub-domain context for traceability.
4. **Sub-domain Navigation:** Agent queries can be scoped by sub-domain (e.g., “Order Fulfillment”), narrowing mapping search to relevant ontology sections.

Example Schema

Table	Key Fields	Purpose
Ontology_Entities	entity_id, name, description, type, embedding, subdomain	Stores ontology classes/properties with vectors and context
Schema_Entities	schema_id, name, type, description, embedding	Digital asset schema classes/properties
Entity_Mappings	mapping_id, source_entity_id, target_entity_id, confidence, type, status, evidence	Stores class-to-class mappings with context and validation
Subdomain_Index	subdomain_id, name, entities	Allows filtering and contextual search for agent queries

Sales Operations Sub-domains

Subset of “Level 1” sub-domains for the Domain (Sales Operations) can include:

- Lead Management
- Quote & Pricing
- Contract Administration
- Order Processing
- Sales Analytics
- Commission Management

Each sub-domain should have its own section of the ontology and mappings, supporting modular and scalable expansion.
design specification for integrating domain-specific ontologies with vector databases and agentic AI, built to improve accuracy, relevancy, and recall. The standard references ISO, IEEE,

and OMG specifications to ensure technical rigor and interoperability.

Design Specification: Sustainable Semantic Scaffold for Agentic AI and Vector Databases

Purpose

The specification defines a modular architecture for domain-centric ontological integration with vector databases in support of agentic AI applications. The goal is to enable robust scenario-mapping, semantic search, high recall, and precision, leveraging international standards to maximize interoperability and sustainability.

Key Steps for Integration

- Use the OMG Ontology Definition Metamodel (ODM) for designing conceptual models and mapping UML/ER schemas to OWL and RDF/JSON-LD, guided by ISO/IEC/IEEE 24765 nomenclature for consistent terminology.
- Transform ontology classes/properties into vector embeddings using formalized graph-based sequence models mapped according to recommended OMG and W3C vocabularies, extending DCAT for metadata.
- Employ the OMG Distributed Ontology, Model and Specification Language (DOL) to unify models across heterogeneous data sources, supporting cross-domain interoperability and ontology linking.
- Ensure conformance by specifying two conformance levels:
    1. *Specification-level*: Applications formally import the ontologies (owl:imports) and resolve logical consistency.
    2. *Linked Data-level*: Applications reference one or more ontologies and use standard OMG/ISO URIs.
- Maintain modularity and reuse across deployments by referencing ISO standards for naming conventions, versioning, and documentation.

Practical Strategies to Improve Search and Relevancy

- Encode ontological data as dense embeddings, mapping classes, relations, and instances into vector representations using IEEE-endorsed neural models (such as semantic encoders or graph neural networks), ensuring compatibility with OWL2/JSON-LD.

Semantic Governance
(Vector DB Design Pattern)

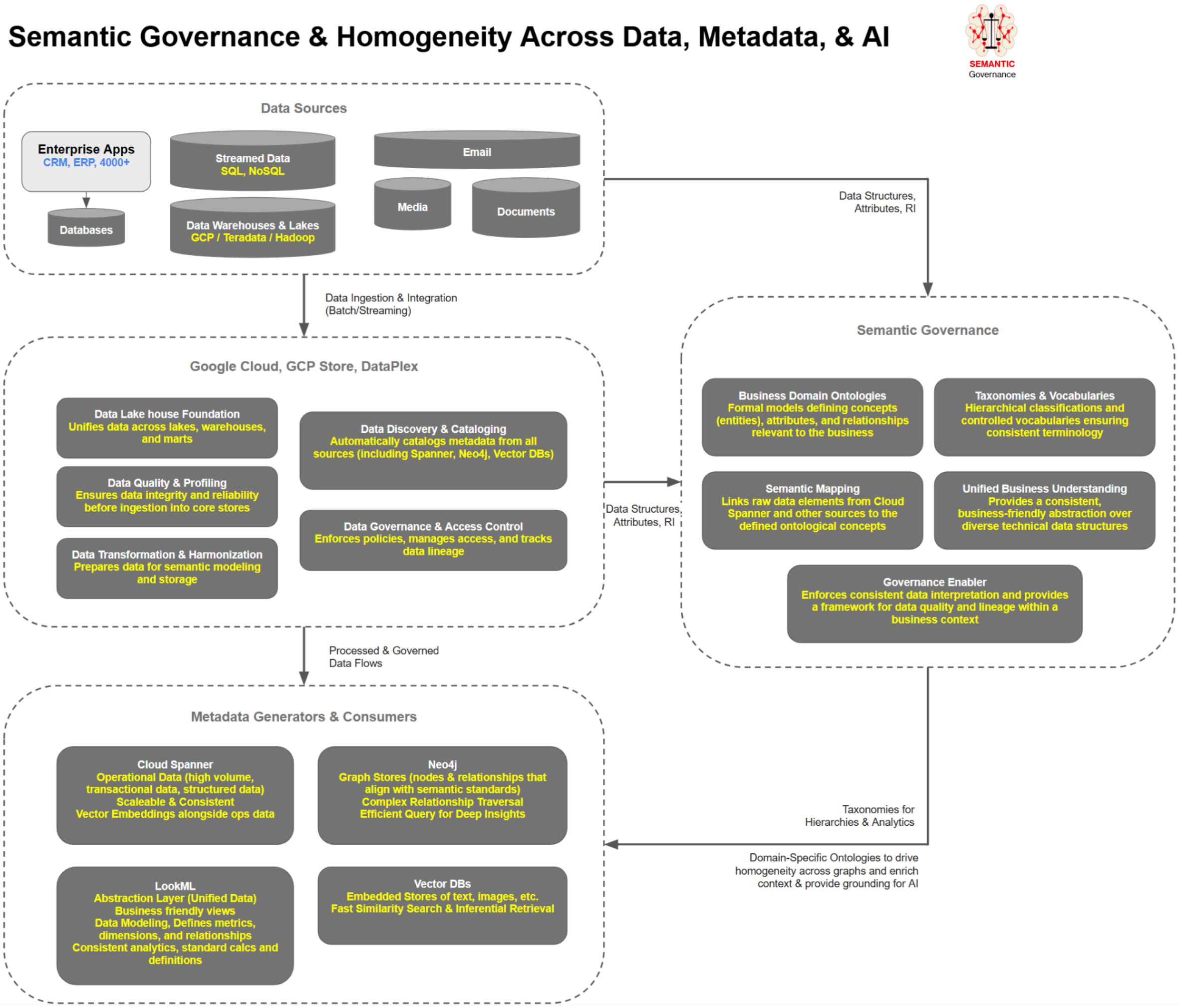
- Use the OMG Data Products Ontology (DPROD) spec to describe data products, metadata, entitlements, and data lineage for search and discovery, leveraging controls for compliance and distributed access.
- Apply RDF and DCAT vocabularies to structure entity metadata, ground semantic search, and facilitate cross-platform discovery and interoperability.
- Enable hybrid search and reranking pipelines that combine symbolic reasoning (OWL2/DOL-based inference) and vector similarity (cosine distance, reranked by ontological rules), guided by scenario-specific ontologies.
- Implement continuous integration and quality assurance for ontological objects following ISO standards for data quality, error reduction, and extensibility.

Implementation Outline

Component	Specification	Standard Ref
Ontology Design	ODM/OWL/RDF/JSON-LD for domain model and semantic mappings	OMG ODM, ISO/IEC/IEEE 24765, W3C RDF/OWL
Data Product Metadata	DCAT extension, DPROD schema for data products and lineage	OMG DPROD, W3C DCAT, ISO naming/versioning
Embedding Procedure	IEEE-endorsed deep graph encoders for semantic vectorization	IEEE ML/AI Guidance, OWL-API, ODM
Vector DB Integration	Schema alignment, rich metadata indexing, linkage to ontological URIs	OMG ODM, ISO/IEC JTC 1/SC 7, IEEE interoperability
Hybrid Retrieval System	Pipeline for vector search reranked by ontological inference	OMG DOL, ISO error reduction standards
Quality & Compliance	Automated tests, versioning, data quality monitoring	ISO documentation, OBO Foundry practices
Scenario Mapping	Participatory ontology engineering, use-case embedding, modular workflows	OMG PSIG, domain-specific ontologies, W3C standards

SG & Homegeneity

Semantic Governance
(Addressing Homogeneity)



SOW Outline

Semantic Governance
SOW Outline (MVP1 Prep)

Caveat for resource(s):

- Need assistance with Design for each of the following breakouts
- This will be a collaborative effort between KE team and resource(s)
- Build activities will involve hands on stubbing out of working services (initial footprints) to get the KE up and running so we can extend and scale up

SOW Outline:

Develop Semantic Services (Processing)

- Mapping Ontologies (overview) + Inferencing Rules
- Semantic Services (Extend Vocabularies)
    - 🌕 Core Vocabularies
    - 🌕 Domain-Specific Vocabularies
- Semantic Services (Extend Taxonomies)
    - 🌕 Core Taxonomies
    - 🌕 Domain-Specific Taxonomies
- Semantic Services (Extend Ontologies)
    - 🌕 Core Ontologies
    - 🌕 Domain-Specific Ontologies
    - 🌕 Mapping Domain-Specific Ontologies:
        - inter-domain mapping
        - intra-domain mapping
        - Digital Asset Structures (RDF Ontologies)
        - Graph mapping

Develop Semantic Services (Outbound)

- Search as a Service
    - 🌕 Get - Simple parms submission from a UX
    - 🌕 Get - Phrases submitted from a UX
    - 🌕 Get - Questions submitted from a UX
    - 🌕 Put - list of criteria for building a relational query (one per table)
    - 🌕 Put - list of criteria for prompt to guide conversation (focus & clarification)
    - 🌕 Put - error management for handling invalid responses or insufficient data
- Semantic Services (Search Results Handoff)

Semantic Governance
SOW Outline (MVP1 Prep)

- Semantic Services (Search Pre-Processor)
- Extend Search for Marketplace (Meaning)
- Extend Search for Marketplace (Relevancy)
    - Semantic Services (Search Persona Relevancy)
    - Semantic Services (Search Contextual Relevancy)

Establish Operating Model for Semantic Governance

- Incorporate KE Standards into operating model
- Align with better practices (industry & experience)
- Leverage SKL to capture Lifecycle artifacts for all components
- Define Monitoring & Measurement for Semantic Governance

Timeline for Deliverable(s):

- Processing Services
- Outbound Services
- Governance Model

Need to have a working set of services (inbound, processing, outbound) and governance in place by End of 3rd Quarter. Ideally, if funds are available we would like to have 3 resources to tackle the priorities in parallel with cross-pollination of design and collaboration throughout.

Assumptions:

- 3rd Party Resources have sufficient independence, leadership, and architecture experience to take the point on design and build efforts.
- DAG-AS will be able to establish our inbound services sufficient to ramp up the ingestion of data structures
- Priorities in order:
    - Processing Services - need to get mapping and inferencing up and running
    - Outbound Services - sufficient to enable a service call from Marketplace
    - Governance Model - lifecycle e2e monitoring, staging, reporting, publishing

Desired Outcomes:

- Processing Services support the mapping of ingested structures against our ontologies.
- Ontologies are dynamically extended during ingestion and processing

Semantic Governance
SOW Outline (MVP1 Prep)

- Outbound Services are published and support simple integration with Marketplace
    - 🌕 Ability to capture basic tokens, boolean logic, and prompt
    - 🌕 Translation of prior point (inputs) into domain-specific graph queries
    - 🌕 Ability to support federated graph queries
    - 🌕 Ability to handoff conditional logic & filtering for downstream orchestration
        - Manual handoff initially - TBD

Design + Success Criteria:

- Reference Success Criteria and Design Doc

SOW (MVP1 Prep)

Semantic Governance
SOW (MVP1 Prep)

Statement of Work: Semantic Services Development & Governance Framework

Project Title: Semantic Services Enablement and Governance Implementation

Date: May 20, 2025

Client: KE Team

Prepared For: [Client Organization Name/Stakeholder - Placeholder]

Prepared By: [Service Provider/Resource(s) - Placeholder]

1. Introduction & Background

This Statement of Work (SOW) outlines the plan for developing and implementing a suite of Semantic Services and establishing a corresponding Operating Model for Semantic Governance. The primary goal is to enhance the Knowledge Enterprise (KE) by enabling sophisticated data interpretation, mapping, inferencing, and search capabilities. These services will facilitate the ingestion, processing, and outbound utilization of data, with an initial focus on integration with the Marketplace.

This initiative recognizes the need for a collaborative approach between the KE team and the engaged resource(s), particularly in the design phase. The build activities will involve practical, hands-on development to create initial, functional service footprints that the KE team can subsequently extend and scale.

2. Project Goals & Objectives

- **Develop robust Semantic Processing Services:** To accurately map ingested data structures against defined ontologies, dynamically extend these ontologies, and apply inferencing rules.
- **Implement effective Semantic Outbound Services:** To provide advanced search capabilities ("Search as a Service") for various user interactions and integrate seamlessly with the Marketplace.
- **Establish a comprehensive Operating Model for Semantic Governance:** To ensure the consistency, quality, and lifecycle management of all semantic components.
- **Deliver a working set of services and governance model by the end of Q3.**

3. Scope of Work

Semantic Governance
SOW (MVP1 Prep)

The scope of this project is divided into three primary workstreams: Semantic Services (Processing), Semantic Services (Outbound), and Semantic Governance Operating Model.

3.1. Develop Semantic Services (Processing)

This workstream focuses on the services required to interpret, map, and enrich data as it enters or is processed within the Knowledge Enterprise.

Caveat for resource(s):

- Design for each of the following breakouts will require assistance and collaborative effort between the KE team and resource(s).
- Build activities will involve hands-on stubbing out of working services (initial footprints) to get the KE up and running for future extension and scaling.

3.1.1. Mapping Ontologies (Overview) + Inferencing Rules

* **Activity:** Define and document the overarching strategy for mapping diverse data sources to the KE's core and domain-specific ontologies.

* **Activity:** Develop and implement a foundational set of inferencing rules (e.g., subsumption, equivalence, disjointness, custom rules) to enable automated reasoning and knowledge discovery from the mapped data. This includes:
    * Rule identification workshops with KE subject matter experts.
    * Rule formalization using appropriate rule languages (e.g., SWRL, SHACL, or custom rule engines).
    * Initial rule testing and validation against sample datasets.

* **Deliverable:** Mapping Strategy Document; Initial Inferencing Rule Set (documented and stubbed in the system).

3.1.2. Semantic Services (Extend Vocabularies)

* **3.1.2.1. Core Vocabularies:**

    * **Activity:** Identify, define, and document foundational vocabularies (e.g., based on Dublin Core, SKOS, FOAF, or custom-defined core concepts) essential for broad interoperability across the KE.

    * **Activity:** Develop services/APIs to manage (create, read, update, delete - CRUD) and extend these core vocabularies dynamically as new concepts are encountered during data ingestion or processing.

    * **Deliverable:** Documented Core Vocabularies; Vocabulary Extension Service (stubbed API).

* **3.1.2.2. Domain-Specific Vocabularies:**

* **Activity:** Identify, define, and document vocabularies specific to key business domains within the KE (e.g., product, customer, finance, research).

* **Activity:** Develop services/APIs to manage and extend these domain-specific vocabularies, ensuring alignment with core vocabularies and domain expert input.

* **Deliverable:** Documented Domain-Specific Vocabularies (for selected initial domains); Domain Vocabulary Extension Service (stubbed API).

3.1.3. Semantic Services (Extend Taxonomies)

* **3.1.3.1. Core Taxonomies:**

    * **Activity:** Develop or adopt foundational hierarchical classification systems (taxonomies) that provide a structured way to organize core concepts within the KE.

    * **Activity:** Develop services/APIs for managing and extending these core taxonomies, including establishing relationships (e.g., broader/narrower terms).

    * **Deliverable:** Defined Core Taxonomies; Taxonomy Extension Service (stubbed API).

* **3.1.3.2. Domain-Specific Taxonomies:**

    * **Activity:** Develop hierarchical classification systems tailored to specific business domains, enabling more granular organization and retrieval of domain-specific information.

    * **Activity:** Develop services/APIs for managing and extending these domain-specific taxonomies, ensuring consistency with core taxonomies and domain requirements.

    * **Deliverable:** Defined Domain-Specific Taxonomies (for selected initial domains); Domain Taxonomy Extension Service (stubbed API).

3.1.4. Semantic Services (Extend Ontologies)

* **3.1.4.1. Core Ontologies:**

    * **Activity:** Define, model, and implement core ontologies that represent fundamental entities, properties, and relationships across the KE (e.g., building upon existing standards like BFO, or developing a custom foundational ontology).

    * **Activity:** Develop services/APIs to manage and dynamically extend these core ontologies, allowing for the addition of new classes, properties, and relationships as the KE evolves.

* **Deliverable:** Documented Core Ontologies (OWL/RDF or chosen formalism); Ontology Extension Service (stubbed API for core ontologies).

* **3.1.4.2. Domain-Specific Ontologies:**

  * **Activity:** Define, model, and implement ontologies tailored to specific business domains, capturing the nuanced semantics, entities, and relationships within those domains.

  * **Activity:** Develop services/APIs to manage and dynamically extend these domain-specific ontologies, ensuring alignment with core ontologies and facilitating deeper domain understanding.

  * **Deliverable:** Documented Domain-Specific Ontologies (for selected initial domains); Domain Ontology Extension Service (stubbed API).

* **3.1.4.3. Domain Mapping Ontologies:**

  * **Activity:** Develop specialized ontologies and associated services focused on mediating and aligning different semantic models.

  * **Sub-Activity (inter-domain mapping):** Create mappings between different domain-specific ontologies within the KE to facilitate cross-domain analysis and data integration. This includes defining equivalent classes, properties, and relationships.

  * **Sub-Activity (intra-domain mapping):** Create mappings within a single domain ontology to reconcile variations in terminology or representation that may arise from different sources or perspectives.

  * **Sub-Activity (data asset mapping):** Develop services to map incoming raw data structures (e.g., database schemas, CSV headers, API response fields) to the concepts and properties defined in the KE's ontologies. This will involve creating transformation rules or using mapping languages (e.g., R2RML, SPARQL-Generate).

  * **Sub-Activity (Graph mapping):** Define and implement strategies for mapping and transforming data between different graph models or representations, if applicable (e.g., property graphs to RDF graphs).

  * **Deliverable:** Domain Mapping Ontology Definitions; Mapping Services (stubbed APIs for inter-domain, intra-domain, data asset, and graph mapping).

3.2. Develop Semantic Services (Outbound)

This workstream focuses on services that enable users and other systems (like the Marketplace) to consume and interact with the knowledge managed by the KE.

Semantic Governance
SOW (MVP1 Prep)

Caveat for resource(s):

- Design for each of the following breakouts will require assistance and collaborative effort between the KE team and resource(s).
- Build activities will involve hands-on stubbing out of working services (initial footprints) to get the KE up and running for future extension and scaling.

3.2.1. Search as a Service

* **3.2.1.1. Get - Simple Parameters Submission from a UX:**

 * **Activity:** Design and develop an API endpoint that accepts simple keyword-based search queries and structured parameters (e.g., filters, facets) from a user interface.

 * **Activity:** Implement basic query processing, including tokenization and matching against indexed semantic data.

 * **Deliverable:** Simple Search API (stubbed and documented).

* **3.2.1.2. Get - Phrases Submitted from a UX:**

 * **Activity:** Design and develop an API endpoint that can process natural language phrases, identifying key concepts and relationships for more nuanced search.

 * **Activity:** Incorporate techniques for phrase recognition and potentially basic intent detection.

 * **Deliverable:** Phrase-Based Search API (stubbed and documented).

* **3.2.1.3. Get - Questions Submitted from a UX (Natural Language Query - NLQ):**

 * **Activity:** Design and develop an API endpoint capable of interpreting natural language questions (e.g., "What are the key products in the X category related to Y?").

 * **Activity:** Implement mechanisms to parse questions, identify entities and intents, and translate them into formal queries against the semantic backend (e.g., SPARQL queries).

 * **Deliverable:** Question-Answering Search API (stubbed and documented).

* **3.2.1.4. Put - List of Criteria for Building a Relational Query (One Per Table):**

 * **Activity:** Design and develop an API endpoint that accepts a structured list of criteria (e.g., field, operator, value) intended for constructing queries against underlying relational data sources, guided by semantic understanding.

* **Activity:** This service will act as an intermediary, translating semantic search intent into parameters suitable for relational database queries where appropriate (e.g., for federated search scenarios).

* **Deliverable:** Relational Query Criteria API (stubbed and documented). *

3.2.1.5. Put - List of Criteria for Prompt to Guide Conversation (Focus & Clarification):

* **Activity:** Design and develop an API endpoint that accepts criteria to generate or refine prompts for conversational AI systems or guided search interactions.

* **Activity:** This service will leverage semantic context to help focus conversations, suggest clarifying questions, or guide users towards relevant information.

* **Deliverable:** Conversational Prompt Guidance API (stubbed and documented).

* **3.2.1.6. Put - Error Management for Handling Invalid Responses or Insufficient Data:**

* **Activity:** Design and develop an API endpoint or mechanism within the search services to manage and communicate errors gracefully.

* **Activity:** This includes handling scenarios like invalid search queries, no results found, ambiguous queries, or insufficient data to fulfill a request, providing meaningful feedback to the calling application/UX.

* **Deliverable:** Error Management API/Specification (documented).

3.2.2. Semantic Services (Search Results Handoff)

* **Activity:** Define and implement a standardized format and protocol for handing off search results to consuming applications (e.g., the Marketplace).

* **Activity:** This includes specifying the structure of result sets, metadata, snippets, links to source data, and any associated semantic annotations or confidence scores.

* **Activity:** Initially, this may be a manual handoff process for validation, with a plan for automation.

* **Deliverable:** Search Results Handoff Specification; Initial (manual) Handoff Process Document.

3.2.3. Semantic Services (Search Pre-Processor)

* **Activity:** Design and develop a service that acts as a pre-processor for incoming search queries.

* **Activity:** This service will perform tasks such as query understanding, disambiguation, expansion (e.g., using synonyms from vocabularies, broader/narrower terms from taxonomies), and entity recognition before the query is passed to the core search engine or graph database.

* **Deliverable:** Search Pre-Processor Service (stubbed API and functional description).

3.2.4. Extend Search for Marketplace (Meaning)

* **Activity:** Enhance search capabilities specifically for the Marketplace by incorporating deeper semantic understanding.

* **Activity:** This involves leveraging ontologies and vocabularies to interpret user intent beyond keywords, understanding the meaning behind search terms in the context of Marketplace offerings.

* **Deliverable:** Enhanced Marketplace Search (Meaning) Design Document; Initial implementation stubs.

3.2.5. Extend Search for Marketplace (Relevancy)

* **Activity:** Improve the relevancy ranking of search results within the Marketplace by utilizing semantic relationships and context.

* **Activity:** Develop algorithms or configure search platforms to prioritize results based on semantic similarity, user context, and business rules defined within the KE.

* **Deliverable:** Enhanced Marketplace Search (Relevancy) Design Document; Initial implementation stubs.

3.2.6. Semantic Services (Search Persona Relevancy)

* **Activity:** Design and develop mechanisms to tailor search results based on user personas or roles.

* **Activity:** This involves defining user personas, associating them with specific interests or information needs (potentially captured semantically), and adjusting search relevancy accordingly.

* **Deliverable:** Persona-Based Relevancy Design Document; Service stubs for persona integration.

3.2.7. Semantic Services (Search Contextual Relevancy)

* **Activity:** Design and develop services to enhance search relevancy by considering the broader context of the user's session, task, or historical interactions.

* **Activity:** This could involve leveraging session data, project context, or other contextual cues to refine search results and provide more pertinent information.

* **Deliverable:** Contextual Relevancy Design Document; Service stubs for context integration.

3.3. Establish Operating Model for Semantic Governance

This workstream focuses on defining the processes, roles, responsibilities, and tools for managing the lifecycle of semantic assets.

3.3.1. Incorporate KE Standards into Operating Model

* **Activity:** Review existing KE standards (data quality, security, architecture, etc.) and define processes to ensure all semantic components (vocabularies, taxonomies, ontologies, rules) adhere to these standards.

* **Activity:** Develop guidelines for the creation, modification, and deprecation of semantic assets in line with KE policies.

* **Deliverable:** Semantic Governance Standards Integration Plan.

3.3.2. Align with Better Practices (Industry & Experience)

* **Activity:** Research and incorporate industry best practices for semantic governance (e.g., DAMA DMBOK, FAIR principles, W3C standards).

* **Activity:** Leverage the experience of the resources and KE team to tailor these practices to the organization's specific needs and culture.

* **Deliverable:** Best Practices Alignment Report and Recommendations.

3.3.3. Leverage SKL (Semantic Knowledge Lifecycle) to Capture Lifecycle Artifacts for All Components

* **Activity:** Define how a Semantic Knowledge Lifecycle (SKL) management approach will be implemented. This includes defining stages (e.g., ideation, development, testing, deployment, maintenance, retirement) for all semantic components.

* **Activity:** Specify the artifacts to be captured at each stage (e.g., requirements, design documents, models, test cases, version history, change logs, approval records).

* **Activity:** Identify or recommend tools/platforms for managing these SKL artifacts (this may involve existing KE tools or proposing new ones).

* **Deliverable:** SKL Process Definition Document; Artifact Management Plan.

3.3.4. Define Monitoring & Measurement for Semantic Governance

* **Activity:** Identify key performance indicators (KPIs) and metrics to monitor the health, quality, usage, and impact of semantic assets and services.

* **Activity:** Examples include: ontology completeness, vocabulary adoption rates, query performance, data mapping accuracy, governance process compliance.

* **Activity:** Define processes and tools for collecting, analyzing, and reporting on these metrics. This includes defining staging, reporting, and publishing mechanisms for governance information.

* **Deliverable:** Semantic Governance Monitoring & Measurement Framework (including KPIs, reporting processes).

4. Deliverables Summary

A summary of key deliverables (detailed within Scope of Work):

Processing Services:

- Mapping Strategy Document
- Initial Inferencing Rule Set
- Documented Core & Domain-Specific Vocabularies
- Vocabulary Extension Service (stubbed API)
- Defined Core & Domain-Specific Taxonomies
- Taxonomy Extension Service (stubbed API)
- Documented Core & Domain-Specific Ontologies
- Ontology Extension Service (stubbed API for core & domain)
- Domain Mapping Ontology Definitions
- Mapping Services (stubbed APIs for inter-domain, intra-domain, data asset, graph mapping)

Outbound Services:

- Simple Search API (stubbed)
- Phrase-Based Search API (stubbed)
- Question-Answering Search API (stubbed)
- Relational Query Criteria API (stubbed)
- Conversational Prompt Guidance API (stubbed)
- Error Management API/Specification
- Search Results Handoff Specification & Initial Manual Process Document
- Search Pre-Processor Service (stubbed API & functional description)
- Enhanced Marketplace Search (Meaning) Design Document & Stubs
- Enhanced Marketplace Search (Relevancy) Design Document & Stubs
- Persona-Based Relevancy Design Document & Service Stubs
- Contextual Relevancy Design Document & Service Stubs

Governance Model:

- Semantic Governance Standards Integration Plan
- Best Practices Alignment Report and Recommendations

- SKL Process Definition Document & Artifact Management Plan
- Semantic Governance Monitoring & Measurement Framework

5. Timeline for Deliverable(s)

The overarching goal is to have a working set of services (inbound integration assumed via DAG-AS, processing, outbound) and the governance model in place by the **End of Q3 [Year]**.

A high-level phased approach is proposed, assuming the availability of 3 resources to work in parallel with cross-pollination:

- **Phase 1: Foundation & Core Design (Month 1-2)**
    - Detailed design workshops for all Processing and Outbound services.
    - Definition of Core Vocabularies, Taxonomies, and Ontologies.
    - Initial draft of Governance Model framework (Standards, SKL, Monitoring).
    - Stubbing out key Processing Service APIs (Mapping, Vocabulary, Taxonomy, Ontology extensions).
    - Stubbing out foundational Outbound Service APIs (Simple Search, Conversational Search - Prompt Iteration, Pre-processor).
- **Phase 2: Development & Initial Integration (Month 2-3)**
    - Development of initial footprints for Processing Services (focus on mapping and inferencing).
    - Development of initial footprints for Outbound Services (focus on Marketplace integration: Meaning & Relevancy, Handoff).
    - Refinement and detailing of the Governance Model.
    - Begin implementation of SKL artifact capture.
- **Phase 3: Testing, Refinement & Deployment (Month 3 - End of Q3)**
    - Testing of Processing and Outbound services with sample data and Marketplace integration points.
    - Refinement of services based on testing feedback.
    - Full documentation of all services and the Governance Model.
    - Deployment of initial working services.
    - Establishment of monitoring and reporting mechanisms for governance.

Priorities (as stated):

1. **Processing Services:** Focus on getting mapping and inferencing up and running early.
2. **Outbound Services:** Sufficient functionality to enable a service call from Marketplace.
3. **Governance Model:** Lifecycle e2e monitoring, staging, reporting, publishing.

6. Project Team & Responsibilities (Illustrative)

Role	Responsibility	Resource Type/Name
KE Team Lead/Sponsor	Overall project oversight, strategic direction, approvals, internal coordination.	[KE Team]
KE Subject Matter Experts	Provide domain knowledge, input for vocabularies/taxonomies/ontologies, validate rules and mappings.	[KE Team]
Lead Semantic Architect	Lead design of semantic services and governance, ensure architectural coherence, guide build efforts.	[Resource 1]
Semantic Developer 1	Focus on Processing Services development (mapping, inferencing, ontology/vocabulary/taxonomy extensions).	[Resource 2]
Semantic Developer 2	Focus on Outbound Services development (Search as a Service, Marketplace integration, relevancy enhancements).	[Resource 3]
Governance Specialist	(Could be shared or part of Architect role) Focus on developing the Semantic Governance Operating Model.	[Resource 1/TBD]
DAG-AS Team Contact	Liaison for inbound service integration and data structure ingestion.	[DAG-AS Team]

7. Assumptions

- **Resource Availability:** Three (3) 3rd Party Resources will be available as outlined, possessing sufficient independence, leadership, and architecture experience to lead design and build efforts collaboratively with the KE team.
- **KE Team Collaboration:** Active participation and timely input from the KE team (including SMEs) for design, validation, and feedback.

- **Inbound Services (DAG-AS):** The DAG-AS team will establish and provide functional inbound services sufficient to ramp up the ingestion of data structures required by the semantic processing services. Clear interface specifications for inbound data will be available.
- **Technology Stack:** Access to and decisions on the necessary technology stack (e.g., ontology editors, triple stores/graph databases, rule engines, API gateways, development environments) will be made in a timely manner. If specific technologies are already in place, access will be provided.
- **Development & Test Environments:** Suitable development and testing environments will be available or provisioned.
- **Access to Data:** Sample data and access to relevant data sources will be provided for development and testing purposes.
- **Marketplace Integration Point:** A clear point of contact and technical specifications for integrating Outbound Services with the Marketplace will be available.

8. Desired Outcomes (Reiteration from Outline)

- **Processing Services Support:** Processing Services effectively support the mapping of ingested data structures against KE ontologies.
- **Dynamic Ontology Extension:** Ontologies are dynamically extended (new concepts, relationships) during data ingestion and processing based on defined rules and new information.
- **Published Outbound Services:** Outbound Services are published and support simple, effective integration with the Marketplace.
- **Input Capture:** Ability to capture basic tokens, boolean logic, and prompts as inputs for search and conversational guidance.
- **Query Translation:** Translation of these inputs into domain-specific graph queries.
- **Federated Query Support:** Ability to support federated graph queries (potentially spanning multiple underlying graph stores or data sources).
- **Conditional Logic Handoff:** Ability to hand off conditional logic and filtering requirements for downstream orchestration by other systems.
- **Initial Manual Handoff:** Search results handoff will initially be manual for validation, with a plan for future automation (TBD).

9. Pricing & Payment Schedule

[Placeholder: This section would typically detail the costs associated with the resources, payment terms, and schedule. This needs to be determined based on resource rates and project duration.]

Example:

- Resource Rates:
  - 🌕 Lead Semantic Architect: $[Rate]/hour
  - 🌕 Semantic Developer: $[Rate]/hour

- Estimated Hours: [Total estimated hours per phase/deliverable]
- Payment Schedule: [e.g., Monthly invoicing based on hours worked, milestone-based payments]

10. Acceptance Criteria

Each deliverable will be considered complete and accepted upon:

- Formal review and approval by the KE Team Lead/Sponsor.
- Demonstration that the deliverable meets the requirements and functionalities outlined in this SOW.
- For service stubs: Successful API call and response according to specification.
- For initial footprints: Demonstration of core functionality with sample data.
- For documentation: Completeness and clarity as agreed upon.

11. SOW Amendments

Any changes or deviations from this SOW must be documented in a formal Change Request, reviewed, and approved by both the Client and the Service Provider before implementation.

This expanded SOW provides a more detailed framework. You'll want to review and refine this with your team and the resources involved, especially around specific technical choices, detailed task breakdowns within phases, and the pricing section.